

Artifact 3: Attack Surface Register

TokenStream CRA risk assessment table. This register identifies interfaces, endpoint or port exposure, actors, authentication, default exposure posture, trust-boundary crossings, sensitive assets, and risk-relevant notes.

interface	endpoint/port	actor	auth	exposed by default	trust boundary crossed	sensitive assets	notes
Orchestrator client API	Host port 8004; GET /health; GET /v1/health; GET /v1/models; POST /v1/chat/completions; POST /v1/rag/query; POST /v1/rag/lookup	Client application backend; optional local chat UI	Bearer machine key when ORCHESTRATOR_API_KEYS_JSON, ORCHESTRATOR_API_KEYS_PATH, or ORCHESTRATOR_API_KEY is configured; scopes include models:list, chat:invoke, rag:query, tools:use	Yes, docker-compose publishes 8004:8004	External / user-controlled -> TokenStream self-hosted runtime	Machine API keys, prompts, chat messages, retrieved context, model choices, policy identifiers, tool results, generated responses	Primary application-facing surface. Treat unauthenticated local development mode as unsuitable for shared or public networks.
Orchestrator internal reload API	Host port 8004; POST /v1/internal/reload	dev-ui / config_auth administrative control plane	ORCHESTRATOR_RELOAD_TOKEN	Yes, because 8004 is published; endpoint is intended for internal control use	Administrative control plane -> TokenStream runtime	Runtime snapshots, provider registry, policy registry, API key registry, RAG settings, MCP settings	Do not expose reload token to client applications. Network filtering is recommended even when token auth is configured.
Browser admin UI	Host port 8010; /; /index.html; /admin; /admin/{path}	Human administrator / operator	Browser session after /v1/auth/login; local development bootstrap admin may be enabled by CONFIG_AUTH_DEV_BOOTS TRAP_ADMIN	Yes, docker-compose publishes 8010:8010	External / user-controlled -> Administrative control plane	Session cookie, admin account state, provider records, policies, corpora, sources, jobs, users, machine key creation flow	Administrative surface, not an end-user UI. Do not expose publicly with default or bootstrap credentials.
Management API	Host port 8010; /v1/auth/*; /v1/management/* including providers, policies, processors, retrieval profiles, API keys, users, RAG settings, corpora, sources, source upload, ingestion jobs, MCP settings	Human administrator; tightly controlled admin automation	config_auth_session cookie for human admin flows; management authorization enforced by config_auth role/session logic	Yes, docker-compose publishes 8010:8010	External / user-controlled -> Administrative control plane	Admin sessions, user records, API key hashes and one-time plaintext key creation responses, provider secret references, policy allowlists, source metadata, uploaded files, runtime configuration	Highest-value inbound control-plane surface. Source upload and key creation paths require special scrutiny.
config_auth internal registry API	Host port 8010 path prefix /internal; /internal/health; /internal/reload; /internal/corpora*; /internal/processors; /internal/retrieval-profiles; /internal/ingestion-jobs*	retrieval-api; ingestion-worker; internal registry clients	CONFIG_AUTH_INTERNAL_TOKEN via internal service headers	Reachable on the published dev-ui port unless network policy blocks /internal from untrusted clients	TokenStream runtime services -> Administrative control plane	Corpus registry, source registry, processors, retrieval profiles, ingestion job state, worker claim and heartbeat state	Internal-only by design. If 8010 is exposed beyond localhost/trusted network, /internal becomes a meaningful attack surface.
Retrieval API	Host port 8000; GET /health; POST /v1/query; POST /v1/lookup; POST /query legacy alias; GET /corpora	orchestrator-api; trusted internal callers	No direct endpoint authentication visible in current retrieval-api implementation; deployment relies on trusted network placement and upstream orchestration	Yes, docker-compose publishes 8000:8000	TokenStream runtime -> RAG / indexing data plane; optionally external caller -> RAG / indexing data plane if port is reachable	Corpus IDs, filters, queries, retrieved chunks, citations, source metadata, lexical/vector/graph index content	Direct access should be an explicit deployment choice. Consider binding to internal network only or adding service auth before non-local use.
Ingestion worker operational API	Container internal port 8002; GET /health; POST /v1/dry-run/chunking; POST /v1/purge/source	dev-ui management API; internal operators; ingestion worker clients	/v1/purge/source requires CONFIG_AUTH_INTERNAL_TOKEN; /v1/dry-run/chunking has no visible token dependency in current implementation	No host port is published in docker-compose	Administrative control plane -> RAG / indexing data plane	Corpus/source IDs, chunk previews, processor config, purge targets, index deletion results	Keep internal. Chunking dry-run can reveal source-derived content and processing behavior.
Source upload/fetch ingress	Management path POST /v1/management/corpora/{corp us_id}/sources/upload on 8010; source fetch by ingestion-worker from URLs/files/repositories/archives	Human admin; corpus/source owner; ingestion-worker fetching external source systems	Admin session for management upload/registration; external fetch depends on configured source access	Upload route is exposed with 8010; fetch egress depends on configured sources	External / user-controlled -> Administrative control plane -> RAG / indexing data plane	Uploaded files, source URLs, corpus metadata, parsed text, chunks, embeddings, source provenance	Main untrusted-content ingress. Assess SSRF, malicious documents, archive handling, parser failures, provenance integrity, and storage quota abuse.
Qdrant vector store	Host port 6333; QDRANT_URL=http://qdrant:6333	retrieval-api; ingestion-worker; any host/network client if exposed	No Qdrant auth configured in docker-compose	Yes, docker-compose publishes 6333:6333	RAG / indexing data plane -> data store; external caller -> data store if host port is reachable	Dense embeddings, vector metadata, collection names, corpus partitioning, retrieval index contents	Publishing Qdrant directly increases index tampering and data disclosure risk. Prefer internal-only exposure.
Object storage API	Host port 9000; RAG_OBJECT_STORAGE_ENDPOINT=minio:9000	dev-ui/config_auth; ingestion-worker; object-storage clients	MINIO_ROOT_USER and MINIO_ROOT_PASSWORD	Yes, docker-compose publishes 9000:9000	Administrative/RAG runtime -> object storage; external caller -> object storage if host port is reachable	Uploaded source objects, fetched source snapshots, bucket contents, object metadata, MinIO root credentials	Avoid root credentials in application paths for production-like deployments. Limit network exposure and use scoped credentials where feasible.

interface	endpoint/port	actor	auth	exposed by default	trust boundary crossed	sensitive assets	notes
Object storage console	Host port 9001; MiniO console	Human operator	MINIO_ROOT_USER and MINIO_ROOT_PASSWORD	Yes, docker-compose publishes 9001:9001	External / user-controlled -> object storage administration	All source objects, buckets, access credentials, storage configuration	Administrative storage console. Should not be publicly reachable.
TEI embedder service	Host port 8080 mapped to container port 80; EMBEDDER_URL=http://tei-embedder:80	ingestion-worker; any host/network client if exposed	No local service auth configured; optional HF_TOKEN only for model download/access to Hugging Face	Yes, docker-compose publishes 8080:80	RAG / indexing data plane -> embedding service; external caller -> embedding service if host port is reachable	Text chunks sent for embedding, embedding model cache, HF token environment value if mishandled	Direct exposure can leak submitted text through logs/telemetry and enable resource exhaustion. Prefer internal-only exposure.
External LLM provider egress	Configured provider base URLs, for example https://api.deepseek.com, OpenAI-compatible APIs, Anthropic APIs, or local Ollama-style endpoints	orchestrator-api	Provider-specific API keys via env:// secret references or provider settings	Outbound only; enabled when provider credentials/configuration are present	TokenStream runtime -> External providers / tools	Provider API keys, prompts, retrieved context, tool results, model outputs, provider routing metadata	Primary confidentiality boundary for prompt/context leakage. Provider allowlists and policy restrictions should be reviewed per deployment.
MCP server/tool egress	Configured MCP_SERVERS entries; streamable HTTP URL often /mcp; legacy SSE uses /sse and /messages	orchestrator-api; configured MCP servers/tools	MCP-server-specific authentication if configured; TokenStream policy allowlists decide tool availability	Outbound only; no MCP server is enabled by default when MCP_SERVERS=[]	TokenStream runtime -> External providers / tools	Tool schemas, tool arguments, tool results, prompts/context passed to tools, MCP credentials or bearer tokens	Treat each MCP tool as a delegated capability. Review tool side effects, SSRF/file/network access, and policy namespace isolation.
Runtime snapshot files	Local volume /runtime; providers.json, api_keys.json, policies.json, rag_settings.json, mcp_servers.json, mcp_settings.json, processors.json, retrieval_profiles.json	config_auth; orchestrator-api; retrieval-api; ingestion-worker; host/container operator	Filesystem/container access controls; generated from authenticated management operations	Mounted into runtime containers; not a network port	Administrative control plane -> TokenStream runtime via shared volume	Provider configuration, policy configuration, API key hashes/material references, RAG and MCP settings, processors and retrieval profiles	Integrity-critical. Tampering can change routing, policy, tools, retrieval, and credential references without calling public APIs.
Local persistent data stores	Volumes/data paths: config_auth_data, qdrant_data, minio_data, /data, /data/lex, config_auth SQLite	TokenStream services; host/container operator; backup/restore processes	Filesystem/container access controls; database/service credentials where applicable	Not a network interface, but mounted/persisted by default	TokenStream runtime -> persistent storage	Config/auth database, source objects, lexical/graph indexes, vector indexes, ingestion artifacts, logs or operational traces	Include in backup, retention, encryption-at-rest, permissions, and incident-response review. OSS local deployments may vary widely.
Optional Open WebUI	Host port 30000 mapped to container port 8080; OPENAI_API_BASE_URL=http://orchestrator-api:8004/v1	Human user/admin of optional local chat UI	WEBUI_AUTH=true; WEBUI_ADMIN_EMAIL and WEBUI_ADMIN_PASSWORD; Open WebUI uses ORCHESTRATOR_API_KEY toward TokenStream	Yes, docker-compose publishes 30000:8080 when open-webui service is started	External / user-controlled -> optional local client -> TokenStream runtime	Open WebUI credentials, prompts, chat history in open_webui_data_bootstrap, TokenStream machine key configured for the UI	Adjacent client surface, not TokenStream admin. Disable or isolate if not required for the assessed deployment.